FH Salzburg
MultiMediaTechnology

Concurrent & Distributed Systems, 1. Klausur am 29.05.2024

Nachname / Vorname: [redacted]

Hinweise: Schreiben Sie leserlich. Antworten Sie knapp und unter Verwendung von Fachterminologie zu begründen. Internetfähige Geräte und schriftliche Unterlagen sind nicht erlaubt (closed books).

Zeitvorgabe: max. Bearbeitungszeit 45 Minuten

15,5

Gesamtpunkte: 16

1. Was versteht man unter Mutual Exclusion und warum ist es wichtig? (2P)

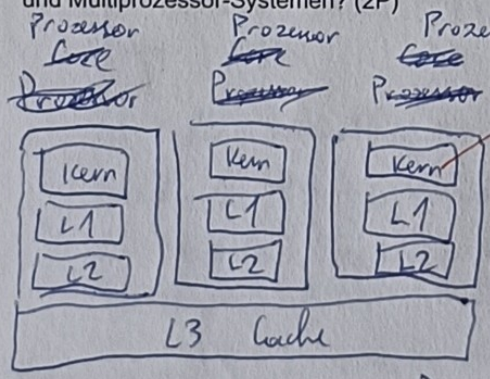
Man versteht darunter, dass man gewisse Codestücke schützt, damit nur ein Thread gleichzeitig diesen Bereich abarbeiten kann.

2

Das braucht man genau dann, wenn 2 Threads auf die gleiche Variable (oder Speicher) gleichzeitig lesend und schreibend zugreifen. Würde man diesen Bereich wo man die Variable nutzt, nicht schützen, würde es zu schwer nachvollziehbaren Fehlern kommen (Data Race, Race Condition, ...), da das Ergebnis dann davon abhängt, wie bzw. wann die Threads ausgeführt werden. Mutual Exclusion schafft man z.B.: mit Monitor, Mutex, Semaphore

2. Wie ist ein Multicore-Prozessor aufgebaut und welche Vorteile bietet er gegenüber Single-Core-Prozessoren und Multiprozessor-Systemen? (2P)

1,5

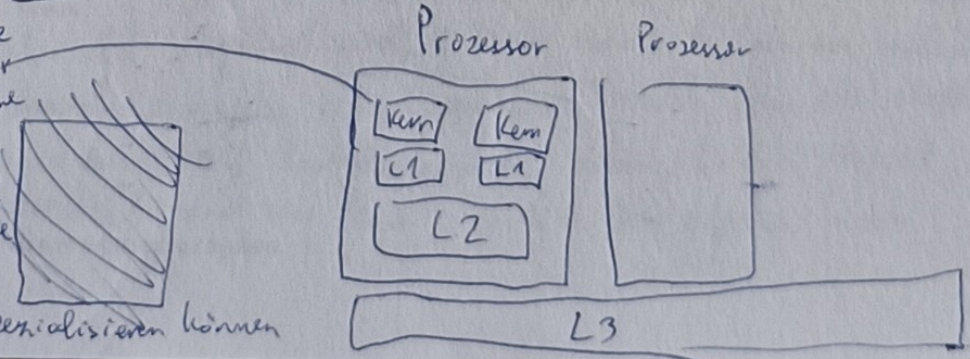


Was ist der Kern?

L1 ... L1 Cache
L2 ... L2 Cache

Single Core Prozessor

Multicore weil Prozessor mehrere Kerne hat, die sich ~~sch~~ evt. auf unterschiedliche Aufgaben spezialisieren können



3. Definieren Sie "Amdahls Gesetz" und diskutieren Sie dessen Auswirkungen für die Parallelisierung von Aufgaben. (2P)

$$s = \frac{1}{f + \frac{1-f}{p}}$$

s beschreibt um wie viel die Aufgabe schneller ausgeführt wird, wenn man sie auf mehrere Threads/Prozesse auslagert und parallel ausführen lässt.

p... Anzahl prozesse/Threads

2 f... Anteil an sequentiell abgearbeiteten Code (Aufgabe aufteilen und Threads zusammen führen gehört auch dazu). Wert zwischen 0 und 1

1-f... Anteil an parallel abgearbeiteten Code

Dadurch kann man ermitteln, ob es sich auszahlt den Algorithmus parallel auszuführen und die Komplexität von parallelen Programmieren einzufügen, oder ob man sich das sparen kann

4. Beantworten sie die folgenden Fragen anhand dieses Codebeispiels.

```
public void Method()
{
    static int i = 0;
    int j = 0;
    object o = new object():
}
```

a. Wo liegen die Variablen **i** und **j** im Speicher? (0,5P)
b. Was ist **o** und wo liegt diese im Speicher? (0,5P)
c. Kann ein Data Race beim Zugriff auf **o** mit mehreren Threads entstehen? Begründung? (1P)

2

a) j liegt am Stack, da int ein Value Type ist und innerhalb einer Methode angelegt wurde

i ist zwar ein Value Type, liegt aber am Heap, da es eine statische variable ist, und daher länger im Speicher liegen muss als die Methode selbst. Um das zu gewährleisten muss die Variable am Heap liegen

b) o ist die Variable die, die Speicheradresse auf den Referenztyp im Speicher speichert. ⇒ Die Variable o liegt am Stack, die eigentlichen Daten des Objects liegen aber am Heap und o verweist nur auf diesen Speicher

c) Wenn o nicht in einer Methode sondern als Member oder statische Variable angelegt wird, Denn dadurch können mehrere Methoden darauf zugreifen bzw. mehrere Threads, wenn die Methoden parallel laufen. In diesem Fall kommt es nicht dazu, da das Objekt in der Methode erstellt und am Ende wieder freigegeben wird.

5. Erläutern Sie die Funktionsweise eines **Monitors** in folgender Implementierung des Producer/Consumer Problems. Kommentieren Sie die relevanten Bereiche im Code und gehen Sie auch auf die einzelnen Methoden des Monitors ein. (2P)

2

```
using System;
using System.Threading;

class Program {
    static int[] buffer = new int[10];
    static int count = 0;
    static int inIndex = 0;
    static int outIndex = 0;

    static void Main(string[] args) {
        Thread producer = new Thread(Produce);
        Thread consumer = new Thread(Consume);

        producer.Start();
        consumer.Start();

        producer.Join();
        consumer.Join();
    }

    static void Produce() {
        for (int i = 0; i < 100; i++) {
            lock (buffer) {
                while (count == buffer.Length) {
                    Monitor.Wait(buffer);
                }

                buffer[inIndex] = i;
                inIndex = (inIndex + 1) % buffer.Length;
                count++;

                Monitor.Pulse(buffer);
            }
        }
    }

    static void Consume() {
        for (int i = 0; i < 100; i++) {
            lock (buffer) {
                while (count == 0) {
                    Monitor.Wait(buffer);
                }

                int value = buffer[outIndex];
                outIndex = (outIndex + 1) % buffer.Length;
                count--;

                Console.WriteLine("Consumed: " + value);

                Monitor.Pulse(buffer);
            }
        }
    }
}
```

Monitor

Monitor

Buffer wird gesperrt, damit schreib operationen nur von einem Thread ausgeführt werden

Ist der Buffer voll, gibt man den lock frei und wartet darauf, dass jemand den Buffer ~~den~~ leert. Bis dahin wird der Thread blockiert

War der Buffer nicht voll, wurde etwas erstellt. Daher wird hier Pulse aufgerufen, um alle Threads zu benachrichtigen, dass sich etwas am State des Buffers verändert hat.

Der Consumer Thread der vom Scheduler ausgewählt wurde blockiert den Bereich wieder und führt den Code aus. Alle anderen Consumer finden den Buffer dann wieder leer vor und warten wieder (solcher die while schleife)

Selbes Prinzip wie beim Producer.

6. Das nachfolgende Programm simuliert eine Bar mit Türsteher. Es sollen maximal 4 Personen die Bar betreten dürfen. Die restlichen Personen müssen draußen warten, bis wieder Plätze in der Bar frei werden. Ergänzen Sie das Programm mit einer **SemaphoreSlim** damit es die geforderte Funktionalität aufweist. (2P)

2

```
class BarSimulation
{
    static Semaphore Slim s = new Semaphore Slim (4,4);
    static void Main(string[] args)
    {
        // Simulate 10 persons trying to enter the bar.
        for (int i = 1; i <= 10; i++)
        {
            int personId = i;
            Thread personThread = new Thread(() => EnterBar(personId));
            personThread.Start();
        }
        Console.ReadLine();
    }

    static void EnterBar(int personId)
    {
        Console.WriteLine($"Person {personId} is trying to enter the bar.");
        s.WaitOne();
        Console.WriteLine($"Person {personId} has entered the bar.");
        Thread.Sleep(1000); // Simulate the person staying in the bar for some time.
        s.Release();
        Console.WriteLine($"Person {personId} is leaving the bar.");
    }
}
```

Semaphore wird betreten

platz in der Semaphore wird freigegeben

7. Schreiben Sie zu diesem Client einen passenden Server. (2P)

2

```
class Program {
    static void Main(string[] args) {
        try
        {
            using var client = new UdpClient();

            var message = "Hello, server!";
            var bytes = Encoding.UTF8.GetBytes(message);
            var endPoint = new IPEndPoint(IPAddress.Loopback, 12345);
            client.Send(bytes, bytes.Length, endPoint);

            var received = client.Receive(ref endPoint);
            var response = Encoding.UTF8.GetString(received);
        }
        catch (Exception e)
        {
            Console.WriteLine($"Error: {e.Message}");
        }
    }
}
```

Server

```
class ~~Client~~ {
    static void Main() {
        try {
            using var listener = new UdpClient(new IPEndPoint(IPAdress.Any, 12345));
            var endpoint = new IPEndPoint(IPAdress.Loopback, 12345);
            byte[] bytes = listener.Receive(ref endpoint);
            var message = Encoding.UTF8.GetBytes(message);
            var bytes2 = Encoding.UTF8.GetBytes("Hello Client");
            listener.Send(bytes2, bytes2.Length, endpoint);
        } catch(Exception e) { ..... }
    }
}
}
```

Using schließt Connection automatisch wenn Object gelöscht wird

vermutlich schreibt die Methode hier die Daten in die Variable und wir müssen sie gar nicht initialisieren

listener.RemoteEndPoint

Fehler Handling

8. Eine Software für Musikverwaltung bietet das nachfolgende API an. Der Service soll mittels gRPC für Zugriffe über das Netzwerk erweitert werden. Beschreiben Sie das Serviceinterface mit Hilfe eines Protofiles. (2P)

```
Record SearchByTitleName (String title);
Record SearchByArtistName (String artist);

class Record {
     String albumName;
     int year;
     String genre;
}
```

2

```
syntax "proto3";

service RecordService {
   rpc SearchByTitleName (TitleSearchRequest) returns (Record) {};
   rpc SearchByArtistName (ArtistSearchRequest) returns (Record) {};
}

message Record {
   string albumName = 1;
   int32 year       = 2;
   string genre     = 3;
}
message TitleSearchRequest {
   string title = 1;
}
message ArtistSearchRequest {
   string artist = 1;
}
```